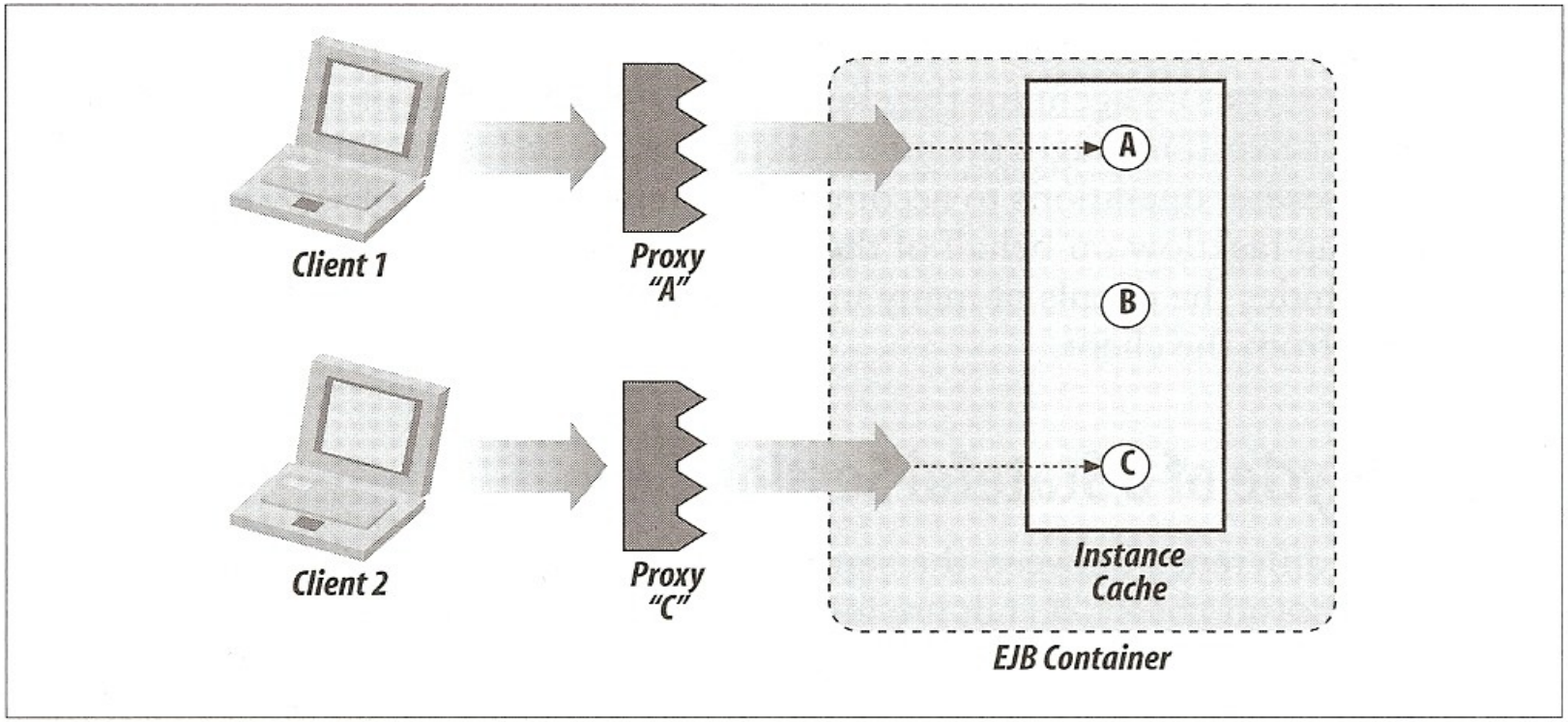
Stateful Session Beans

Chapter 6

The Stateful Session Bean

- Stateful session beans are dedicated to one client for the life of the bean instance
  - Paired with EJBObject (the proxy) for life
  - Unlike stateless ones
- Bean instance can keep conversational state about object
  - Methods can be interdependent
  - Methods can affect subsequent method calls
- This allows thinning the client and moving state information to the server
- Not for long-term storage, may time out
  - Not saved when server shuts down

Client Relation with EJB Container and Bean



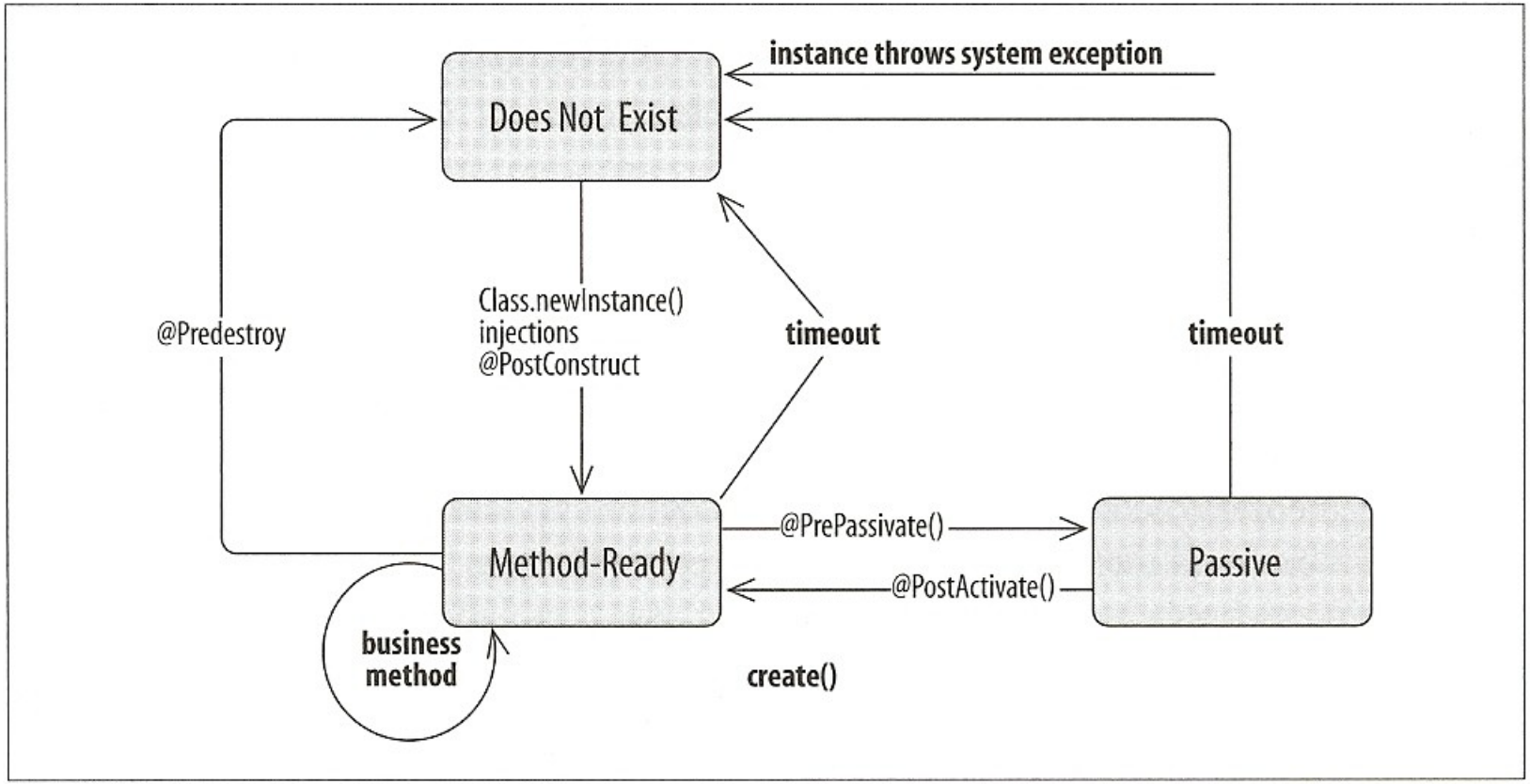
Client View: Using Session Beans

- Always consider what the user wants from a session bean
  - Should provided needed business services
  - Session bean often used to provide data to display, or process results from interface
- To use session bean, need to find it
  - If running in application server (e.g. servlets) use injection (`@EJB`)
  - Running outside application server, use JNDI lookup
  - Each injection/lookup creates new session bean
  - Can also inject component with `@Inject`

SFSB Life Cycle

- SFSBs do not use instance pooling
  - SLSBs do
- If bean is idle, container can remove it from memory and garbage collect instance
  - bean will be
    - *passivated* to preserve conversational state and
    - *activated* if it is used again
- Beans can implement `jakarta.ejb.SessionSynchronization`
  - Notify bean of its participation in transactions
    - Can cache DB data across method calls
  - Discussed in the Transactions chapter 17

Session Bean Life Cycle (Without SessionSynchronization)



Method-Ready State

- Client calls method
  - Initiates transition from Does Not Exist to Method-Ready state
  - Container creates new instance, injects dependencies, assigns instance to client
- In Method-Ready state bean can receive method calls and process them
- Bean can leave Method-Ready state if
  - Container needs memory (calls @PrePassivate method)
  - Client calls @Remove method (bean destroyed)
  - Bean times out (bean destroyed after any transaction completes)

Passivated State

- Passivation saves bean's conversational state
- Bean's conversational state consists of
  - Bean's primitive values
  - Bean's serializable objects
  - Following special types:
    - `jakarta.ejb.SessionContext`
    - `jakarta.transaction.UserTransaction`
    - `javax.naming.Context` (when uses JNDI ENC)
    - `jakarta.persistence.EntityManager`
    - `jakarta.persistence.EntityManagerFactory`
    - References to managed resource factories such as `javax.sql.DataSource`
    - References to other EJBs

Passivation

- When bean about to be passivated, @PrePassivate method called
- Bean should
  - Close open resources
  - Set nontransient nonserializable fields to null
  - (fields labeled `transient` not saved)
- Container saves fields on previous slide
  - If all fields fall into categories in previous slide, do not need @PrePassivate method
- Bean instance is serialized and saved

Activation

- Client calls method of passivated EJB
  - Container activates its instance
- Activation reverses the process of passivation
  - Bean deserialized
  - SessionContext reference, bean references, managed resource factories restored
  - Transient fields unpredictible
  - @PostActivate method should
    - Restore open resources
    - Initialize transient and nonserializable fields

System Exception Thrown by Bean Method

- Recall: System exception is
  - Unchecked exception
  - Not annotated with @ApplicationException
- Container invalidates stateful session bean EJB object
  - Destroys bean instance
  - Does not call @PreDestroy method
  - Moves instance to Does not Exist state

Nested Stateful Session Beans

- If a stateful session bean has another stateful session bean injected
  - containing bean owns the session
  - when containing bean is removed, so is contained one
- See later for issues regarding shared Extended persistence contexts

Example: FileTransferEJB

- File Transfer Protocol (FTP) defines a way to transfer files over a network
- Specification requires conversational state
  - Not all information present in each request
- For implementation as session bean, need stateful one
- Example implements subset
  - pwd, mkdir, cd
- Connection is not serializable and needs handling for passivation/activation

Business Interface

```
public interface FileTransferCommonBusiness {
    void mkdir(String directory)
                    throws IllegalStateException;

    void cd(String directory)
                    throws IllegalStateException;

    String pwd() throws IllegalStateException;

    void disconnect();

    void connect() throws IllegalStateException;
}
```

Remote Interface

- Method endSession is only for use with EJB client
  - so not included in business interface

```
public interface FileTransferRemoteBusiness extends
FileTransferCommonBusiness {
    /**
     * Ends the current session
     * will result in a SFSB @Remove call
     */
    void endSession();
}
```

Bean Implementation Class

- Bean annotated as `@Stateful` with `@Remote` interface
  - Name specified in `@Stateful` so global JNDI name known
- Connect host and port hard coded
  - Better to set up from configuration parameters
- Internal state is Apache commons ftp client and working directory
  - ftp client reinitialized upon activation
  - working directory will be changed to after activation

Startup and Activation

- Need to create FTP client, connect, login and cd to working directory
  - If working directory is set
- Annotate `connect()` method
  - `@PostConstruct` for bean creation
  - `@PostActivate` for activation
    - After passivation

Passivation and Shutdown

- Need to clean up bean
  - Logout and disconnect
  - Set client field to null so can be garbage collected
- Annotate `disconnect()` method
  - `@PrePassivate` for passivation
  - `@PreDestroy` before bean is removed

Unit Testing with JUnit

- For unit test we test FileTransferBean as POJO with JUnit 5
  - No container
    - No injections, lifecycle callbacks
  - Can simulate @PostConstruct by hand
    - Use @BeforeEach to annotate setup method
    - Use @AfterEach to annotate cleanup method

Testing Passivation Logic

- For this, need to simulate container
- Call pre-passivate method explicitly
- Serialize bean to ByteArrayOutputStream
- Unserialize bean
- Call post-activate method explicitly
- Check ftp client working
  - In same working directory as before passivation process

Integration Testing

- Also need to test inside EJB container
- Can do with real container
  - Write special test driver
  - Look up EJB via JNDI
  - TestNG can be used here
- Can also use Arquillian
  - Much faster
  - Packages up test and uses embedded container

Integration Tests

- Should test things that require container
- Make sure session bean is removed when done
  - Calling business method after that should throw exception
- Make sure multiple beans do not interfere with each other
- Can also test business methods

Summary

- State can be stored many places
  - On client, in web session, in stateful session bean, in entity bean, or in database
  - Best place to store state is a design tradeoff
  - Pick one and then test for performance bottlenecks
- Stateful session beans good alternative to CDI beans
  - When need the cross-cutting services of container
  - See earlier lectures for list